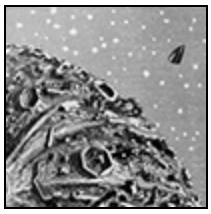


Chapter 10. Using Types for Safety and Inspection

What's a type? As a first approximation, let's say the *type* of a variable specifies the kind of values you can assign to it—for example, integers or lists of strings. When Guido van Rossum created Python, most popular programming languages fell into two camps when it came to types: static and dynamic typing.

Statically typed languages, like C++, require you to declare the types of variables upfront (unless the compiler is smart enough to infer them automatically). In exchange, compilers ensure a variable only ever holds compatible values. That eliminates entire classes of bugs. It also enables optimizations: compilers know how much space the variable needs to store its values.

Dynamically typed languages break with this paradigm: they let you assign any value to any variable. Scripting languages like JavaScript and Perl even convert values implicitly—say, from strings to numbers. This radically speeds up the process of writing code. It also gives you more leeway to shoot yourself into the foot.

Python is a dynamically typed language, yet it chose a middle ground between the opposing camps. Let's demonstrate its approach with an example:

```
import math

number = input("Enter a number: ")
number = float(number)
result = math.sqrt(number)
```

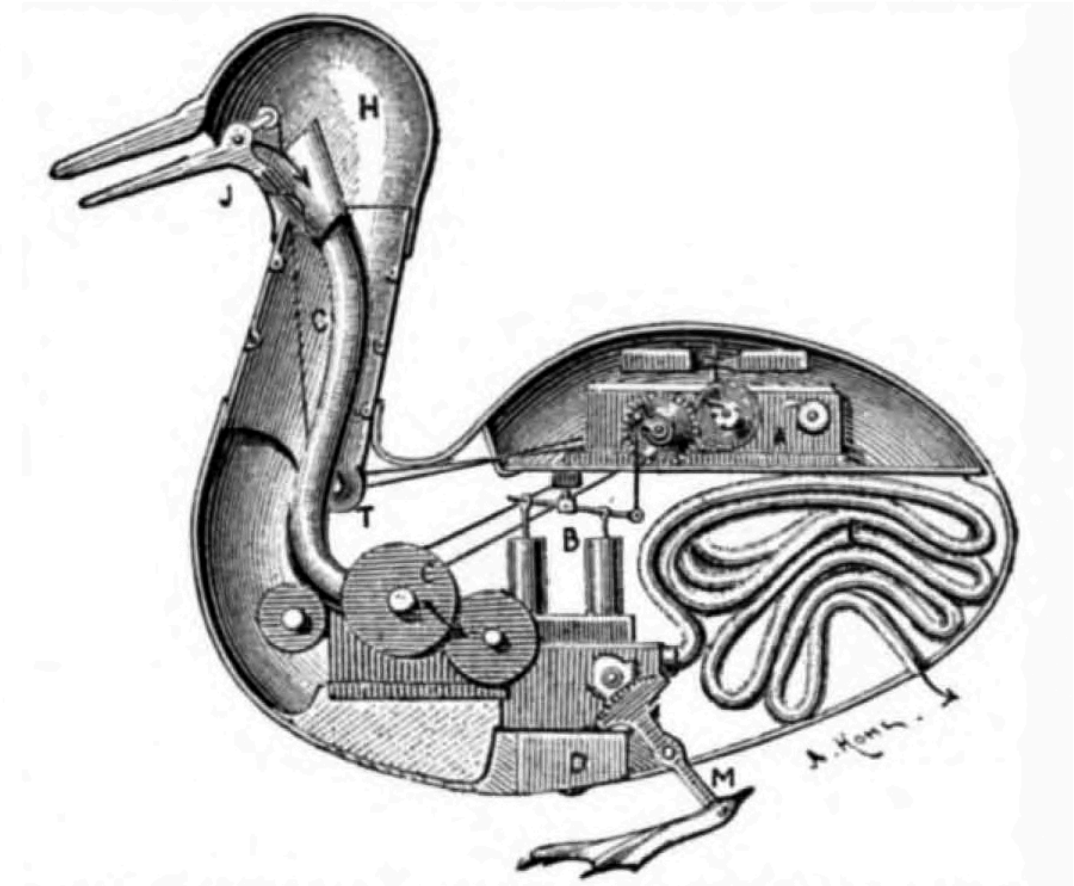
```
print(f"The square root of {number} is {result}.")
```

In Python, a variable is just a name for a value. Variables don't have types —*values* do. The program associates the same name, `number`, first with a value of type `str`, then with a value of type `float`. But unlike Perl and similar languages, Python never converts the values behind your back, in eager anticipation of your wishes:

```
>>> math.sqrt("1.21")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: must be real number, not str
```

Python isn't as forgiving as some of its contemporaries, but consider two limitations of this type check. First, you won't see a `TypeError` until you run the offending code. Second, the Python interpreter doesn't raise the error—the library function checks explicitly if something other than an integer or floating-point number was passed.

Most Python functions don't check the types of their arguments at all. Instead, they simply invoke the operations they expect their arguments to provide. Fundamentally, the type of a Python object doesn't matter as long as its behavior is correct. Taking inspiration from Vaucanson's mechanical duck from the times of Louis XV, this approach is known as *duck typing*: "If it looks like a duck and quacks like a duck, then it must be a duck."



INTERIOR OF VAUCANSON'S AUTOMATIC DUCK.

A, clockwork; *B*, pump; *C*, mill for grinding grain; *F*, intestinal tube; *J*, bill; *H*, head; *M*, feet.

As an example, consider the *join* operation in concurrent code. This operation lets you wait until some background work completes, "joining" the threads of control back together, as it were. Example 10-1 defines a duck-typed function that invokes `join` on a number of tasks, waiting for each in turn.

Example 10-1. Duck typing in action

```
def join_all(joinables):
    for task in joinables:
        task.join()
```

You can use this function with `Thread` or `Process` from the standard `threading` or `multiprocessing` modules—or with any other object that has a `join` method with the correct signature. (You can't use it with

strings because `str.join` takes an argument—an iterable of strings.) Duck typing means that these classes don't need a common base class to benefit from reuse. All the types need is a `join` method with the correct signature.

Duck typing is great because the function and its callers can evolve fairly independently—a property known as *loose coupling*. Without duck typing, a function argument has to implement an explicit interface that specifies its behavior. Python gives you loose coupling for free: you can pass literally anything, as long as it satisfies the expected behavior.

Unfortunately, this freedom can make some functions hard to understand.

If you've ever had to read an entire codebase to grasp the purpose of a few lines within it, you know what I mean: it can be impossible to understand a Python function in isolation. Sometimes, the only way to decipher what's going on is to look at its callers, their callers, and so on (Example 10-2).

Example 10-2. An obscure function

```
def _send(objects, sender):
    """send them with the sender..."""
    for obj in objects[0].get_all():
        if p := obj.get_parent():
            sender.run(p)
        elif obj is not None and obj._next is not None:
            _send_next_object(obj._next, sender)
```

Writing out the argument and return types of the function—its type signature—dramatically reduces the amount of context you need to understand the function. Traditionally, people have done so by listing the types in a docstring. Unfortunately, docstrings are often missing, incomplete, or incorrect. More importantly, there was no formal language for describing the types in a precise and verifiable way. And without tooling to enforce the type signatures, they amounted to little more than wishful thinking.

While this kind of problem is mildly annoying in a codebase with a few hundred lines of code, it quickly turns into an existential threat when you're dealing with many millions of lines of code. Python codebases of this size are common at companies like Google, Meta, Microsoft, and Dropbox, all of which sponsored the development of static type checkers in the 2010s. A *static type checker* is a tool that verifies the type safety of a program without running it. In other words, it checks that the program doesn't perform operations on values that don't support those operations.

To some extent, type checkers can deduce the type of a function or variable automatically, using a process called *type inference*. They become much more powerful when you give programmers a way to specify types explicitly in their code. By the middle of the last decade, and thanks in particular to the foundational work of Jukka Lehtosalo and collaborators,[1] the Python language acquired a way to express the types of functions and variables in source code, called *type annotations* (Example 10-3).

Example 10-3. A function with type annotations

```
def format_lines(lines: list[str], indent: int = 0) -> str:
    prefix = " " * indent
    return "\n".join(f"{prefix}{line}" for line in lines)
```

Type annotations have become the foundation of a rich ecosystem of developer tools and libraries.

On their own, type annotations mostly don't affect the runtime behavior of a program. The interpreter doesn't check that assignments are compatible with the annotated type; it merely stores the annotation inside the special `__annotations__` attribute of the containing module, class, or function. While this incurs a small overhead at runtime, it means you can inspect type annotations at runtime to do exciting stuff—say, construct your domain objects from values transmitted on the wire, without any boilerplate.

One of the most important uses of type annotations, though, doesn't happen at runtime: static type checkers, like mypy, use them to verify the correctness of your code without running it.

Benefits and Costs of Type Annotations

You don't have to use type annotations yourself to benefit from them. Type annotations are available for the standard library and many PyPI packages. Static type checkers can warn you when you use a module incorrectly, including when a breaking change in the library means your code no longer works with that library—and type checkers can warn you *before* you run your code.

Editors and IDEs leverage type annotations to give you a better coding experience, with auto-completion, tooltips, and class browsers. You can also inspect type annotations at runtime, unlocking powerful features such as data validation and serialization.

If you use type annotations in your own code, you reap more benefits. First, you're also a user of your own functions, classes, and modules—so all the benefits previously mentioned apply, like auto-completion and type checking. Additionally, you'll find it easier to reason about your code, refactor it without introducing subtle bugs, and build a clean software architecture. When you are a library author, typing lets you specify an interface contract your users can rely on, while you're free to evolve the implementation.

Even a decade after their introduction, type annotations aren't free of controversy—maybe understandably so, given Python's proud stance as a dynamically typed language. Adding types to existing code poses similar challenges as introducing unit tests to a codebase that wasn't written with testing in mind. Just as you may need to refactor for testability, you may need to refactor for "typability"—replacing deeply nested primitive types and highly dynamic objects with simpler and more predictable types. You'll likely find it worth the effort.

Another challenge is the rapid evolution of the Python typing language. These days, Python type annotations are governed by the Typing Council, which maintains a single, living specification of the typing language.[2] You can expect this specification to undergo more substantial changes over the coming years. While typed Python code needs to navigate this evolution, the typing language makes no exceptions to Python's backward compatibility policy.

In this chapter, you'll learn how to verify the type safety of your Python programs using the static type checker mypy and the runtime type checker Typeguard. You'll also see how runtime inspection of type annotations can greatly enhance the functionality of your programs. But first, let's take a look at the typing language that has evolved within Python over the past decade.

A Brief Tour of Python's Typing Language

Try out the small examples in this section on one of the type-checker playgrounds:

- mypy Playground
- Pyright Playground
- Pyre Playground

Variable Annotations

You can annotate a variable with the type of values that it may be assigned during the course of the program. The syntax for such type annotations consists of the variable name, a colon, and a type:

```
answer: int = 42
```

Besides the simple built-in types like `bool`, `int`, `float`, `str`, or `bytes`, you can also use standard container types in type annotations, such as

`list`, `tuple`, `set`, or `dict`. For example, here's how you might initialize a variable used to store a list of lines read from a file:

```
lines: list[str] = []
```

While the previous example was somewhat redundant, this one provides actual value: without the type annotation, the type checker can't deduce that you want to store strings in the list.

The built-in containers are examples of *generic types*—types that take one or more arguments. Here's an example of a dictionary mapping strings to integers. The two arguments of `dict` specify the key and value types, respectively:

```
fruits: dict[str, int] = {
    "banana": 3,
    "apple": 2,
    "orange": 1,
}
```

Tuples are a bit special, because they come in two flavors. Tuples can be a combination of a fixed number of types, such as a pair of a string and int:

```
pair: tuple[str, int] = ("banana", 3)
```

Another example of this flavor holds coordinates in three-dimensional space:

```
coordinates: tuple[float, float, float] = (4.5, 0.1, 3.2)
```

The other common use of tuples is as an immutable sequence of arbitrary length. To accomodate for this, the typing language lets you write an ellipsis for zero or more items of the same type. For example, here's a tuple that can hold any number of integers (including none at all):

```
numbers: tuple[int, ...] = (1, 2, 3, 4, 5)
```

Any class you define in your own Python code is also a type:

```
class Parrot:
    pass

class NorwegianBlue(Parrot):
    pass

parrot: Parrot = NorwegianBlue()
```

The Subtype Relation

The types on both sides of an assignment aren't necessarily identical. In the example above, you assign a `NorwegianBlue` value to a `Parrot` variable. This works because a Norwegian Blue is a kind of parrot—or technically speaking, because `NorwegianBlue` is a subclass of `Parrot`.

In general, the Python typing language requires that the type on the right-hand side of a variable assignment be a *subtype* of the type on the left-hand side. A prime example of the subtype relation is the relationship of a subclass to its base class, like `NorwegianBlue` and `Parrot`.

However, subtypes are a more general concept than subclasses. For example, a tuple of ints (like `numbers`) is a subtype of a tuple of objects. Unions, introduced in the next section, are yet another example.

TIP

Typing rules also permit assignments if the type on the right is *consistent* with that on the left. This lets you assign an `int` to a `float`, even though `int` isn't derived from `float`. The `Any` type is consistent with any other type (see "Gradual Typing").

Union Types

You can combine two types using the pipe operator (`|`) to construct a *union type*, which is a type whose values range over all the values of its constituent types. For example, you can use it for a user ID that's either numeric or a string:

```
user_id: int | str = "nobody"  # or 65534
```

Arguably the most important use of the union type is for "optional" values, where the missing value is encoded by `None`. Here's an example where a description is read from a *README*, provided that the file exists:

```
description: str | None = None

if readme.exists():
    description = readme.read_text()
```

Union types are another example of the subtype relation: each type involved in the union is a subtype of the union. For example, `str` and `None` are each subtypes of the union type `str | None`.

I skipped over `None` when discussing the built-in types. Strictly speaking, `None` is a value, not a type. The type of `None` is called `NoneType`, and it's available from the standard `types` module. For convenience, Python lets you write `None` in annotations to refer to the type as well.

Tony Hoare, a British computer scientist who has made foundational contributions to many programming languages, famously called the invention of null references, or `None`, his "billion-dollar mistake," due to the number of bugs they've caused since their introduction in `ALGOL` in 1965. If you've ever seen a system crash with an error like the following, you may agree with him. (Python raises this error if you attempt to access an attribute on an object that's in fact `None`.)

```
AttributeError: 'NoneType' object has no attribute '...'
```

The good news is that type checkers can warn you when you're using a variable that's potentially `None`. This can greatly reduce the risk of crashes in production systems.

How do you tell the type checker that your use of `description` is fine? Generally, you should just check that the variable isn't `None`. The type checker will pick up on this and allow you to use the variable:

```
if description is not None:
    for line in description.splitlines():
        print(f"    {line}")
```

There are several methods for *type narrowing*, as this technique is known. I won't discuss them all in detail here. As a rule of thumb, the control flow must reach the line in question only when the value has the right type—and type checkers must be able to infer this fact from the source code. For example, you could also use the `assert` keyword with a built-in function like `isinstance`:

```
assert isinstance(description, str)
for line in description.splitlines():
    ...
```

If you already *know* that the value has the right type, you can help out the type checker using the `cast` function from the `typing` module:

```
description_str = cast(str, description)
for line in description_str.splitlines():
    ...
```

At runtime, the `cast` function just returns its second argument. Unlike `isinstance`, it works with arbitrary type annotations.

Gradual Typing

In Python, every type ultimately derives from `object`. This is true for user-defined classes and primitive types alike, even for types like `int` or `None`. In other words, `object` is a *universal supertype* in Python—you can assign literally anything to a variable of this type.

This may sound kind of powerful, but it really isn't. In terms of behavior, `object` is the smallest common denominator of all Python values, so there's precious little you can do with it, as far as type checkers are concerned:

```
number: object = 2
print(number + number)  # error: Unsupported left operand type for +
```

There's another type in Python that, like `object`, can hold any value. It's called `Any` (for obvious reasons), and it's available from the standard `typing` module. When it comes to behavior, `Any` is `object`'s polar opposite. You can invoke any operation on a value of type `Any`—conceptually, it behaves like the intersection of all possible types. `Any` serves as an escape hatch that lets you opt out of type checking for a piece of code:

```
from typing import Any

number: Any = NorwegianBlue()
print(number + number)  # valid, but crashes at runtime!
```

In the first example, the `object` type results in a false positive: the code works at runtime, but type checkers will reject it. In the second example, the `Any` type results in a false negative: the code crashes at runtime, but type checkers won't flag it.

WARNING

When you're working in typed Python code, watch out for `Any`. It can disable type checking to a surprising degree. For example, if you access attributes or invoke operations on `Any` values, you'll end up with more `Any` values.

The `Any` type is Python's hat trick that lets you restrict type checking to portions of a codebase—formally known as *gradual typing*. In variable assignments and function calls, `Any` is consistent with every other type, and every type is consistent with it.

There are at least a couple of reasons why gradual typing is valuable. First, Python existed without type annotations for two decades, and Python's governing body has no plans to make type annotations obligatory. Therefore, typed and untyped Python will coexist for the foreseeable future. Second, Python's strength comes in part from its ability to be highly dynamic where needed—for example, Python makes it easy to assemble or even modify classes on the fly. In some cases, it's hard (or outright impossible) to apply strict types to such highly dynamic code.[3]

Function Annotations

As you may recall from Example 10-3, type annotations for function arguments look quite similar to those for variables. Return types, on the other hand, are introduced with a right arrow instead of a colon—after all, the colon already introduces the function body in Python. For example, here's a type-annotated function for adding two numbers:

```
def add(a: int, b: int) -> int:
    return a + b
```

Python functions that don't include a `return` statement implicitly return `None`. You might expect return type annotations to be optional in this case, too. This is not the case! As a general rule, always specify the return type when annotating a function:

```
def greet(name: str) -> None:
    print(f"Hello, {name}")
```

Type checkers assume that a function without a return type returns `Any`. Likewise, function parameters without annotations default to `Any`.

Effectively, this disables type checking for the function—exactly the behavior you'd want in a world with large bodies of untyped Python code.

Let's look at a slightly more involved function signature:

```
import subprocess
from typing import Any

def run(*args: str, check: bool = True, **kwargs: Any) -> None:
    subprocess.run(args, check=check, **kwargs)
```

Parameters with default arguments, like `check`, use a syntax similar to variable assignments. The `*args` parameter holds the tuple of positional arguments—each argument must be a `str`. The `**kwargs` parameter holds the dictionary of keyword arguments—using `Any` means that the keyword arguments aren't restricted to any specific type.

In Python, you can use `yield` inside a function to define a *generator*, which is an object that produces a series of values you can use in a `for` loop. Generators support some behavior beyond iteration; when used only for iteration, they're known as *iterators*. Here's how you'd write their type:

```
from collections.abc import Iterator

def fibonacci() -> Iterator[int]:
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
```

Functions are first-class citizens in Python. You can assign a function to a variable or pass it to another function—for example, to register a callback. Consequently, Python lets you express the type of a function outside of a function definition. `Callable` is a generic type that takes two arguments—a list of parameter types and the return type:

```
from collections.abc import Callable

Serve = Callable[[Article], str]
```

Annotating Classes

The rules for variable and function annotations also apply in the context of class definitions, where they describe instance variables and methods. You can omit the annotation for the `self` argument in a method. Type checkers can infer instance variables from assignments in a `__init__` method:

```
class Swallow:
    def __init__(self, velocity: float) -> None:
        self.velocity = velocity
```

The standard `dataclasses` module generates the canonical method definitions from the type annotations of any class decorated with `@dataclass`:

```
from dataclasses import dataclass

@dataclass
class Swallow:
    velocity: float
```

The dataclass-style definition isn't just more concise than the handwritten one; it also confers the class additional runtime behavior—such as the ability to compare instances for equality based on their attributes, or to order them.

When you're annotating classes, the problem of forward references often appears. Consider a two-dimensional point, with a method to compute its Euclidean distance from another point:

```
import math
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

    def distance(self, other: Point) -> float:
        dx = self.x - other.x
        dy = self.y - other.y
        return math.sqrt(dx*dx + dy*dy)
```

While type checkers are happy with this definition, the code raises an exception when you run it with the Python interpreter:[4]

```
NameError: name 'Point' is not defined. Did you mean: 'print'?
```

Python doesn't let you use `Point` in the method definition because you're not done defining the class—the name doesn't exist yet. There are several ways to resolve this situation. First, you can write the forward reference as a string to avoid the `NameError`, a technique known as *stringized* annotations:

```
@dataclass
class Point:
    def distance(self, other: "Point") -> float:
        ...
```

Second, you can implicitly stringize all annotations in the current module using the `annotations` future import:

```
from __future__ import annotations

@dataclass
class Point:
    def distance(self, other: Point) -> float:
        ...
```

The third method does not help with all forward references, but it does here. You can use the special `Self` type to refer to the current class:

```
from typing import Self

@dataclass
class Point:
    def distance(self, other: Self) -> float:
        ...
```

Beware of a semantic difference in this third version compared to the earlier ones. If you derived a `SparklyPoint` class from `Point`, `Self` would refer to the derived class rather than the base class. In other words, you wouldn't be able to compute the distance of sparkly points from plain old points.

Type Aliases

You can use the `type` keyword to introduce an alias for a type:[5]

```
type UserID = int | str
```

This technique is useful to make your code self-documenting and to keep it readable when the types become unwieldy, as they sometimes do. Type aliases also let you define types that otherwise would be impossible to express. Consider an inherently recursive data type such as a JSON object:

```
type JSON = None | bool | int | float | str | list[JSON] | dict[str, JSON]
```

Recursive type aliases are another example of forward references. If your Python version doesn't yet support the `type` keyword, you'll need to replace `JSON` with `"JSON"` on the righthand side to avoid a `NameError`.

Generics

As you've seen at the beginning of this section, built-in containers like `list` are generic types. You can also define generic functions and classes yourself, and it's quite straightforward to do so. Consider a function that returns the first item in a list of strings:

```
def first(values: list[str]) -> str:
    for value in values:
        return value
    raise ValueError("empty list")
```

There's no reason to restrict the element type to a string: the logic doesn't depend on it. Let's make the function generic for all types. First, replace `str` with the placeholder `T`. Second, mark the placeholder as a *type variable* by declaring it in square brackets after the function name. (The name `T` is just a convention; you could name it anything.) Additionally, there's no reason to restrict the function to lists, because it works with any type over which you can iterate in a `for` loop—in other words, any *iterable*:

```
from collections.abc import Iterable

def first[T](values: Iterable[T]) -> T:
    for value in values:
        return value
    raise ValueError("no values")
```

Here's how you might use the generic function in your code:

```
fruit: str = first(["banana", "orange", "apple"])
number: int = first({1, 2, 3})
```

You can omit the variable annotations for `fruit` and `number`, by the way—type checkers infer them from the annotation of your generic function.

NOTE

Generics with the `[T]` syntax are supported in Python 3.12+ and the Pyright type checker. If you get an error, omit the `[T]` suffix from `first` and use `TypeVar` from the `typing` module:

```
T = TypeVar("T")
```

Protocols

The `join_all` function from Example 10-1 works with threads, processes, or any other objects you can join. Duck typing makes your functions simple and reusable. But how can you verify the implicit contract between the functions and their callers?

Protocols bridge the gap between duck typing and type annotations. A protocol describes the behavior of an object without requiring the object to inherit from it. It looks somewhat like an *abstract base class*—a base class that doesn't implement any methods:

```
from typing import Protocol

class Joinable(Protocol):
    def join(self) -> None: ...
```

The `Joinable` protocol requires the object to have a `join` method that takes no arguments and returns `None`. The `join_all` function can use the protocol to specify which objects it supports:

```
def join_all(joinables: Iterable[Joinable]) -> None:
    for task in joinables:
        task.join()
```

Remarkably, this piece of code works with standard library types like `Thread` or `Process`, even though they don't have any knowledge of your `Joinable` protocol—a prime example of loose coupling.

This technique is known as *structural subtyping*: it's the internal structure of `Thread` and `Process` that makes them subtypes of `Joinable`. By contrast, *nominal subtyping* requires you to derive the subtype from the supertype explicitly.

Compatibility with Older Python Versions

The description above is based on the latest Python release as of this writing, Python 3.12. Table 10-1 lists typing features that aren't yet available on older Python versions, as well as their replacements in those versions.

Table 10-1. Availability of typing features

Feature	Example	Availability	Replacement
Generics in standard collections	`list[str]`	Python 3.9	`typing.List`
Union operator	`str | int`	Python 3.10	`typing.Union`
Self type	`Self`	Python 3.11	`typing_extensions.Self`
`type` keyword	`type UserID = ...`	Python 3.12	`typing.TypeAlias` (Python 3.10)
Type parameter syntax	`def first[T](...)`	Python 3.12	`typing.TypeVar`

The `typing-extensions` library provides backports for many features not available in older Python versions; see "Automating mypy with Nox".

This concludes our brief tour of Python's typing language. While there's a lot more to typing in Python, I hope that this overview has taught you enough to make deeper forays into the exciting world of typing on your own.

Static Type Checking with mypy

Mypy is a widely used static type checker for Python. A static type checker uses type annotations and type inference to detect bugs in a program without running it. Mypy was the original reference implementation when the typing system was codified in PEP 484. This doesn't mean that mypy is always the first type checker to implement a new feature of the typing language—for example, the `type` keyword was first implemented in Pyright. However, it's certainly a good default choice, and core members of the typing community are involved in its development.

First Steps with mypy

Add mypy to the development dependencies of your project—for example, by adding a `typing` extra:

```
[project.optional-dependencies]
typing = ["mypy>=1.9.0"]
```

You can now install mypy in the project environment:

```
$ uv pip install -e ".[typing]"
```

If you use Poetry, add mypy to your project using `poetry add`:

```
$ poetry add --group=typing "mypy>=1.9.0"
```

Finally, run mypy on the *src* directory of your project:

```
$ py -m mypy src
Success: no issues found in 2 source files
```

Let's type-check some code with a type-related bug. Consider the following program, which passes `None` to a function that expects a string:

```
import textwrap

data = {"title": "Gegenes nostrodamus"}

summary = data.get("extract")
summary = textwrap.fill(summary)
```

If you run mypy on this code, it dutifully reports that the argument in the call to `textwrap.fill` isn't guaranteed to be a string:

```
$ py -m mypy example.py
example.py:5: error: Argument 1 to "fill" has incompatible type "str | None";
  expected "str"  [arg-type]
Found 1 error in 1 file (checked 1 source file)
```

Revisiting the Wikipedia Example

Let's revisit the Wikipedia API client from Example 6-3. In a fictional scenario, sweeping censorship laws have been passed. Depending on the country you're connecting from, the Wikipedia API omits the article summary.

You could store an empty string when this happens. But let's be principled: an empty summary isn't the same as no summary at all. Let's store `None` when the response omits the field.

As a first step, change the `summary` default to `None` instead of an empty string. Use a union type to signal that the field can hold `None` instead of a string:

```
@dataclass
class Article:
    title: str = ""
    summary: str | None = None
```

A few lines below, the `show` function reformats the summary to ensure a line length of 72 characters or fewer:

```
def show(article, file):
    summary = textwrap.fill(article.summary)
    file.write(f"{article.title}\n\n{summary}\n")
```

Presumably, mypy will balk at this error, just like it did above. Yet, when you run it on the file, it's all sunshine. Can you guess why?

```
$ py -m mypy src
Success: no issues found in 2 source files
```

Mypy doesn't complain about the call because the `article` parameter doesn't have a type annotation. It considers `article` to be `Any`, so the expression `article.summary` also becomes `Any`. (`Any` is infectious.) As far as mypy is concerned, that expression can be `str`, `None`, and a pink elephant, all at the same time. This is gradual typing in action, and it's why you should be wary of `Any` types and missing annotations in your code.

You can help mypy detect the error by annotating the parameter as `article: Article`. Try actually fixing the bug, as well—think about how you would handle the case of summaries being `None` in a real program. Here's one way to solve this:

```
def show(article: Article, file):
    if article.summary is not None:
        summary = textwrap.fill(article.summary)
    else:
        summary = "[CENSORED]"
    file.write(f"{article.title}\n\n{summary}\n")
```

Strict Mode

Mypy defaults to gradual typing by treating parameters and return values as `Any` if they don't have type annotations. Turn on strict mode in *pyproject.toml* to opt out of this lenient default:

```
[tool.mypy]
strict = true
```

The `strict` setting changes the defaults of a dozen-odd more fine-grained settings. If you run mypy on the module again, you'll notice that the type checker has become a lot more opinionated about your code. In strict mode, both defining and calling untyped functions will result in an error:[6]

```
$ py -m mypy src
__init__.py:16: error: Function is missing a type annotation
__init__.py:22: error: Function is missing a type annotation
__init__.py:27: error: Function is missing a return type annotation
__init__.py:27: note: Use "-> None" if function does not return a value
__init__.py:28: error: Call to untyped function "fetch" in typed context
__init__.py:29: error: Call to untyped function "show" in typed context
__main__.py:3: error: Call to untyped function "main" in typed context
Found 6 errors in 2 files (checked 2 source files)
```

Example 10-4 shows the module with type annotations and introduces two concepts you haven't seen yet. First, the `Final` annotation marks `API_URL` as a constant—a variable to which you can't assign another value. Second, the `TextIO` type is a file-like object for reading and writing strings (`str`), such as the standard output stream. Otherwise, the type annotations should look fairly familiar.

Example 10-4. The Wikipedia API client with type annotations

```
import json
import sys
import textwrap
import urllib.request
from dataclasses import dataclass
from typing import Final, TextIO

API_URL: Final = "https://en.wikipedia.org/api/rest_v1/page/random/summary"

@dataclass
class Article:
```

```
        title: str = ""
        summary: str = ""

    def fetch(url: str) -> Article:
        with urllib.request.urlopen(url) as response:
            data = json.load(response)
        return Article(data["title"], data["extract"])

    def show(article: Article, file: TextIO) -> None:
        summary = textwrap.fill(article.summary)
        file.write(f"{article.title}\n\n{summary}\n")

    def main() -> None:
        article = fetch(API_URL)
        show(article, sys.stdout)
```

I recommend strict mode for any new Python project because it's much easier to annotate your code as you write it. Strict checks give you more confidence in the correctness of your program because type errors are less likely to be masked by `Any`.

TIP

My other favorite mypy setting in *pyproject.toml* is the `pretty` flag. It displays source snippets and indicates where the error occurred:

```
[tool.mypy]
pretty = true
```

Let mypy's strict mode be your North Star when adding types to an existing Python codebase. Mypy gives you an arsenal of finer- and coarser-grained ways to relax type checking when you're not ready to fix a type error.

Your first line of defense is a special comment of the form `# type: ignore`. Always follow it with the error code in square brackets. For example, here's a line from mypy's previous output with the error code included:

```
__main__.py:3: error: Call to untyped function "main" in typed context
  [no-untyped-call]
```

You can allow this specific call to an untyped function as follows:

```
main()  # type: ignore[no-untyped-call]
```

If you have a module with a large number of untyped calls, you can ignore the error for the entire module using the following stanza in your *pyproject.toml*:

```
[tool.mypy."<module>"] ❶
allow_untyped_calls = true
```

❶ Replace `<module>` with the module that has the untyped calls. Use double quotes if the module's name contains any dots.

You can also ignore an error globally, like this:

```
[tool.mypy]
allow_untyped_calls = true
```

You can even disable all type errors for a given module:

```
[tool.mypy."<module>"]
ignore_errors = true
```

Automating mypy with Nox

Throughout this book, you've automated checks for your projects using Nox. Nox sessions allow you and other contributors to run checks easily and repeatedly during local development, the same way they'd run on a continuous integration (CI) server.

Example 10-5 shows a Nox session for type checking your project with mypy.

Example 10-5. A Nox session for type checking with mypy

```
import nox

@nox.session(python=["3.12", "3.11", "3.10"])
def mypy(session: nox.Session) -> None:
    session.install(".[typing]")
    session.run("mypy", "src")
```

Just like you run the test suite across all supported Python versions, you should also type-check your project on every Python version. This practice is fairly effective at ensuring that your project is compatible with those versions, even when your test suite doesn't exercise that one code path where you forgot about backward compatibility.

NOTE

You can also pass the target version using mypy's `--python-version` option. However, installing the project on each version ensures that mypy checks your project against the correct dependencies. These may not be the same on all Python versions.

Inevitably, as you type-check on multiple versions, you'll get into situations where either the runtime code or the type annotations don't work across all versions. For example, Python 3.9 deprecated `typing.Iterable` in favor of `collections.abc.Iterable`. Use conditional imports based on the Python version, as shown below. Static type checkers recognize Python version checks in your code, and they will base their type checks on the code relevant for the current version:

```
import sys

if sys.version_info >= (3, 9):
    from collections.abc import Iterable
```

```
else:
    from typing import Iterable
```

Another sticking point: typing features not yet available at the low end of your supported Python version range. Fortunately, these often come with backports in a third-party library named `typing-extensions`. For example, Python 3.11 added the useful `Self` annotation, which denotes the currently enclosing class. If you support versions older than that, add `typing-extensions` to your dependencies and import `Self` from there:

```
import sys

if sys.version_info >= (3, 11):
    from typing import Self
else:
    from typing_extensions import Self
```

Distributing Types with Python Packages

You may wonder why the Nox session in Example 10-5 installs the project into mypy's virtual environment. By nature, a static type checker operates on source code; it doesn't run your code. So why install anything but the type checker itself?

To see why this matters, consider the version of the Wikipedia project in Examples 6-5 and 6-14, where you implemented the `show` and `fetch` functions using Rich and `httpx`. How can a type checker validate your use of a specific version of a third-party package?

Rich and `httpx` are, in fact, fully type annotated. They include an empty marker file named *py.typed* next to their source files. When you install the packages into a virtual environment, the marker file allows static type checkers to locate their types.

Many Python packages distribute their types inline with *py.typed* markers. However, other mechanisms for type distribution exist. Knowing them is useful when mypy can't import the types for a package.

For example, the `factory-boy` library doesn't yet ship with types—instead, you need to install a stubs package named `types-factory-boy` from PyPI.[7] A *stubs package* is a Python package containing typing stubs, a special kind of Python source file with a *.pyi* suffix that has only type annotations and no executable code.

If you're entirely out of luck and types for your dependency simply don't exist, disable the mypy error in *pyproject.toml*, like this:

```
[tool.mypy.<package>] ❶
ignore_missing_imports = true
```

❶ Replace `<package>` with the import name of the package.

NOTE

Python's standard library doesn't include type annotations. Type checkers vendor the third-party package `typeshed` for standard library types, so you don't have to worry about supplying them.

Type Checking the Tests

Treat your tests like you would treat any other code. Type checking your tests helps you detect when they use your project, pytest, or testing libraries incorrectly.

TIP

Running mypy on your test suite also type-checks the public API of your project. This can be a good fallback when you're unable to fully type your implementation code for every supported Python version.

Example 10-6 extends the Nox session to type-check your test suite. Install your test dependencies so mypy has access to type information for pytest and friends.

Example 10-6. A Nox session for type checking with mypy

```
nox.options.sessions = ["lint", "mypy", "tests"]

@nox.session(python=["3.12", "3.11", "3.10"])
def mypy(session: nox.Session) -> None:
    session.install(".[typing,tests]")
    session.run("mypy", "src", "tests")
```

The test suite imports your package from the environment. The type checker therefore expects your package to distribute type information. Add an empty *py.typed* marker file to your import package, next to the `__init__` and `__main__` modules (see "Distributing Types with Python Packages").

There isn't anything inherently special about typing a test suite. Recent versions of pytest come with high-quality type annotations. These help when your tests use one of pytest's built-in fixtures. Many test functions don't have arguments and return `None`. Here's a slightly more involved example using a fixture and test from Chapter 6:

```
import io
import pytest
from random_wikipedia_article import Article, show

@pytest.fixture
def file() -> io.StringIO:
    return io.StringIO()

def test_final_newline(article: Article, file: io.StringIO) -> None:
    show(article, file)
    assert file.getvalue().endswith("\n")
```

Finally, let's indulge in a bout of self-referentialism and type-check the *noxfile.py* (Example 10-7). You'll need the `nox` package to validate your use of Nox. I'll use a little trick here: when the session runs, there's already a suitable environment with Nox installed—you're in it! Instead of

creating another environment with Nox, point mypy to the existing one using its `--python-executable` option.

Example 10-7. Type checking the noxfile.py with mypy

```
import sys

@nox.session(python=["3.12", "3.11", "3.10"])
def mypy(session: nox.Session) -> None:
    session.install(".[typing,tests]")
    session.run("mypy", "src", "tests")
    session.run("mypy", f"--python-executable={sys.executable}", "noxfile.py")
```

Inspecting Type Annotations at Runtime

Unlike in TypeScript, where static types are available only during compilation, Python type annotations are also available at runtime. Runtime inspection of type annotations is the foundation for powerful features, and an ecosystem of third-party libraries has evolved around its use.

The interpreter stores the type annotations in a special attribute named `__annotations__` on the enclosing function, class, or module. Don't access this attribute directly, however—consider it part of Python's plumbing. Python deliberately doesn't shield the attribute from you, but it provides a high-level interface that's easy to use correctly: the function `inspect.get_annotations()`.

Let's inspect the type annotations of the `Article` class from Example 10-4:

```
>>> import inspect
>>> inspect.get_annotations(Article)
{'title': <class 'str'>, 'summary': <class 'str'>}
```

Recall that the `fetch` function instantiates the class like this:

```
    return Article(data["title"], data["extract"])
```

If you can instantiate an `Article`, it must have a standard `__init__` method that initializes its attributes. (You can convince yourself of this fact by accessing it in the interactive session.) Where does the method come from?

The Zen of Python[8] says, "Special cases aren't special enough to break the rules." Dataclasses make no exception to this principle: they're plain Python classes without any secret sauce. Given that the class doesn't define the method itself, there's only one possible origin for it: the `@dataclass` class decorator. In fact, the decorator synthesizes the `__init__` method on the fly, along with several other methods, using your type annotations! Don't take my word for it, though. In this section, you're going to write your own miniature `@dataclass` decorator.

WARNING

Don't use this in production! Use the standard `dataclasses` module, or better: the `attrs` library. Attrs is an actively maintained, industry-strength implementation with better performance, a clean API, and additional features, and it directly inspired `dataclasses`.

Writing a @dataclass Decorator

First of all, be a good typing citizen and think about the signature of the `@dataclass` decorator. A class decorator accepts a class and returns it, usually after transforming it in some way, such as by adding a method. In Python, classes are objects you can pass around and manipulate to your liking.

The typing language allows you to refer to, say, the `str` class by writing `type[str]`. You can read this aloud as "the type of a string." (You can't use `str` on its own here. In a type annotation, `str` just refers to an individual string.) A class decorator should work for any class object, though

—it should be generic. Therefore, you'll use a type variable instead of an actual class like `str`:[9]

```
def dataclass[T](cls: type[T]) -> type[T]:
    ...
```

Type checkers need one more bit to properly understand your `@dataclass` decorator: they need to know which methods you're adding to the class and which instance variables they can expect on objects instantiated from the class. Traditionally, you had to write a type checker plugin to infuse this knowledge into the tool. These days, the `@dataclass_transform` marker from the standard library lets you inform type checkers that the class exhibits dataclass-like behavior:

```
from typing import dataclass_transform

@dataclass_transform()
def dataclass[T](cls: type[T]) -> type[T]:
    ...
```

With the function signature out of the way, let's think about how to implement the decorator. You can break this into two steps. First, you'll need to assemble a string with the source code of the `__init__` method, using the type annotations on the dataclass. Second, you can use Python's built-in `exec` function to evaluate that source code in the running program.

You've likely written a few `__init__` methods in your career—they're pure boilerplate. For the `Article` class, the method would look like this:

```
def __init__(self, title: str, summary: str) -> None:
    self.title = title
    self.summary = summary
```

Let's tackle the first step: assembling the source code from the annotations (Example 10-8). Don't fret too much about the parameter types at this point—just use the `__name__` attribute of each parameter type, which will work in many cases.

Example 10-8. Generating the code of the `__init__` method

```
def build_dataclass_init[T](cls: type[T]) -> str: ❶
    annotations = inspect.get_annotations(cls) ❷

    args: list[str] = ["self"] ❸
    body: list[str] = []

    for name, type in annotations.items():
        args.append(f"{name}: {type.__name__}")
        body.append(f"    self.{name} = {name}")

    return "def __init__({}) -> None:\n{}".format(
        ', '.join(args),
        '\n'.join(body),
    )
```

❶ Use a type variable `T` in the signature to make this generic for any class.

❷ Retrieve the annotations of the class as a dictionary of names and types.

❸ The variable annotation is only required for `body`. Most type checkers won't infer that `body` contains strings because it's an empty list at this point. I've annotated both variables for symmetry.

You can now pass the source code to the `exec` built-in. Apart from the source code, this function accepts dictionaries for the global and local variables.

The canonical way to retrieve the global variables is the `globals()` built-in. However, you need to evaluate the source code in the context of the module where the class is defined, rather than the context of your decorator. Python stores the name of that module in the `__module__` attribute of the class, so you can look up the module object in `sys.modules` and retrieve the variables from its `__dict__` attribute (see "The Module Cache"):

```
    globals = sys.modules[cls.__module__].__dict__
```

For the local variables, you can pass an empty dictionary—this is where `exec` will place the method definition. All that's left is to copy the method from the locals dictionary into the class object and return the class. Without further ado, Example 10-9 shows the entire decorator.

Example 10-9. Your own `@dataclass` decorator

```
@dataclass_transform()
def dataclass[T](cls: type[T]) -> type[T]:
    sourcecode = build_dataclass_init(cls)

    globals = sys.modules[cls.__module__].__dict__ ❶
    locals = {}
    exec(sourcecode, globals, locals) ❷

    cls.__init__ = locals["__init__"] ❸
    return cls
```

❶ Retrieve the global variables from the module that defines the class.

❷ This is where the magic happens: let the interpreter compile the generated code on the fly.

❸ Et voilà—the class now has an `__init__` method.

Runtime Type Checking

There's more you can do with types at runtime besides generating class boilerplate. One important example is runtime type checking. To see how useful this technique is, let's take another look at the `fetch` function:

```
def fetch(url: str) -> Article:
    with urllib.request.urlopen(url) as response:
```

```
        data = json.load(response)
    return Article(data["title"], data["extract"])
```

You may have noticed that `fetch` is not type-safe. Nothing guarantees that the Wikipedia API will return a JSON payload of the expected shape. You might object that Wikipedia's OpenAPI specification tells us exactly which data shape to expect from the endpoint. But don't base your static types on assumptions about external systems—unless you're happy with your program crashing when a bug or API change breaks those assumptions.

As you may have guessed, mypy silently passes over this issue, because `json.load` returns `Any`. How can we make the function type-safe? As a first step, let's replace `Any` with the `JSON` type you defined in "Type Aliases":

```
def fetch(url: str) -> Article:
    with urllib.request.urlopen(url) as response:
        data: JSON = json.load(response)
    return Article(data["title"], data["extract"])
```

We haven't fixed the bug, but at least mypy gives us diagnostics now (edited for brevity):

```
$ py -m mypy src
error: Value of type "..." is not indexable
error: No overload variant of "__getitem__" matches argument type "str"
error: Argument 1 to "Article" has incompatible type "..."; expected "str"
error: Invalid index type "str" for "JSON"; expected type "..."
error: Argument 2 to "Article" has incompatible type "..."; expected "str"
Found 5 errors in 1 file (checked 1 source file)
```

Mypy's diagnostics boil down to two separate issues in the function. First, the code indexes `data` without verifying that it's a dictionary. Second, it passes the results to `Article` without making sure they're strings.

Let's check the type of `data` then—it has to be a dictionary with strings under the `title` and `extract` keys. You can express this concisely using

structural pattern matching:

```
def fetch(url: str) -> Article:
    with urllib.request.urlopen(url) as response:
        data: JSON = json.load(response)

    match data:
        case {"title": str(title), "extract": str(extract)}:
            return Article(title, extract)

    raise ValueError("invalid response")
```

Thanks to type narrowing, the runtime type checks also appease mypy—in a way, bridging the worlds of runtime and static type checking. If you'd like to see the runtime type check in action, you can use the test harness from Chapter 6 and modify the HTTP server to return an unexpected response, such as `null` or `"teapot"`.

Serialization and Deserialization with cattrs

The function is type-safe now, but can we do better than this? The validation code duplicates the structure of the `Article` class—you shouldn't need to spell out the types of its fields again. If your application must validate more than one input, the boilerplate can hurt readability and maintainability. It should be possible to assemble articles from JSON objects using only the original type annotations—and it is.

The `cattrs` library provides flexible and type-safe serialization and deserialization for type-annotated classes such as dataclasses and attrs. It's delightfully simple to use—you pass the JSON object and the expected type to its `structure` function and get the assembled object back.[10] There's also a `destructure` function for transforming objects into primitive types for serialization.

For this last iteration on the Wikipedia example, add `cattrs` to your dependencies:

```
[project]
dependencies = ["cattrs>=23.2.3"]
```

Replace the `fetch` function with the three-liner below (don't run this yet, we'll get to the final version in a second):

```
import cattrs

def fetch(url: str) -> Article:
    with urllib.request.urlopen(url) as response:
        data: JSON = json.load(response)
    return cattrs.structure(data, Article)
```

Deserializing `Article` objects is now entirely determined by their type annotations. Besides being clear and concise, this version of the code is type-safe, thanks to the internal runtime checks in `cattrs`.

However, you still need to take care of one complication. The `summary` attribute doesn't match the name of its corresponding JSON field, `extract`. Fortunately, `cattrs` is flexible enough to let you create a custom converter that renames the field on the fly:

```
import cattrs.gen

converter = cattrs.Converter()
converter.register_structure_hook(
    Article,
    cattrs.gen.make_dict_structure_fn(
        Article,
        converter,
        summary=cattrs.gen.override(rename="extract"),
    )
)
```

Finally, use the custom converter in the `fetch` function:

```
def fetch(url: str) -> Article:
    with urllib.request.urlopen(url) as response:
```

```
        data: JSON = json.load(response)
    return converter.structure(data, Article)
```

From a software-architecture perspective, the `cattrs` library has advantages over other popular data-validation libraries. It keeps serialization and deserialization separate from your *models*—the classes at the core of your application that express its problem domain and provide all the business logic. Decoupling the domain model from the data layer gives you architectural flexibility and improves the testability of your code.[11]

There are also practical advantages to the `cattrs` approach. You can serialize the same object in different ways if you need to. It's not intrusive—it doesn't add methods to your objects. And it works with all kinds of types: dataclasses, attrs-classes, named tuples, typed dicts, and even plain type annotations like `tuple[str, int]`.

Runtime Type Checking with Typeguard

Do you find the type unsafety of the original `fetch` function disconcerting? The issue was easy enough to spot in a short script. But how do you find similar issues in a large codebase before they cause problems? After all, you ran mypy in strict mode, and it remained silent.

Static type checkers won't catch every type-related error. In this case, gradual typing obscured the issue—specifically, `json.load` returning `Any`. Real-world code has plenty of situations like this. A library outside of your control might have overly permissive type annotations—or none at all. A bug in your persistence layer might load corrupted objects from disk. Maybe mypy would have caught the issue, but you silenced type errors for the module in question.

Typeguard is a third-party library and pytest plugin for runtime type checking. It can be an invaluable tool for verifying the type safety of your code in situations that elude static type checkers, such as:

Dynamic code

Python code can be highly dynamic, forcing type annotations to be permissive. Your assumptions about the code may be at odds with the concrete types you end up with at runtime.

External systems

Most real-world code eventually crosses the boundary to external systems such as a web service, a database, or the file system. Data you receive from these systems may not have the shape you expect it to. Its format can also unexpectedly change from one day to another.

Third-party libraries

Some of your Python dependencies may not have type annotations, or their type annotations might be incomplete or overly permissive.

Add Typeguard to your dependencies in *pyproject.toml*:

```
[project]
dependencies = ["typeguard>=4.2.1"]
```

Typeguard comes with a function called `check_type`, which you can think of as `isinstance` for arbitrary type annotations. Those annotations can be quite simple—say, a list of floating-point numbers:

```
from typeguard import check_type

numbers = check_type(data, list[float])
```

The checks can also be more elaborate. For example, you can use the `TypedDict` construct to specify the precise shape of a JSON object you've fetched from some external service, such as the keys you expect to find and which types their associated values should have:[1][2]

```
from typing import Any, TypedDict

class Person(TypedDict):
    name: str
    age: int

    @classmethod
    def check(cls, data: Any) -> "Person":
        return check_type(data, Person)
```

Here's how you might use it:

```
>>> Person.check({"name": "Alice", "age": 12})
{'name': 'Alice', 'age': 12}
>>> Person.check({"name": "Carol"})
typeguard.TypeCheckError: dict is missing required key(s): "age"
```

Typeguard also comes with a decorator named `@typechecked`. When used as a function decorator, it instruments the function to check the types of its arguments and return value. When used as a class decorator, it instruments every method in this way. For example, you could slap this decorator onto a function that reads `Person` records from a JSON file:

```
@typechecked
def load_people(path: Path) -> list[Person]:
    with path.open() as io:
        return json.load(io)
```

By default, Typeguard checks only the first item in a collection to reduce runtime overhead. You can change this strategy to check all items in the global configuration object:[13]

```
import typeguard
from typeguard import CollectionCheckStrategy

typeguard.config.collection_check_strategy = CollectionCheckStrategy.ALL_ITEMS
```

Finally, Typeguard comes with an import hook that instruments all functions and methods in a module on import. While you can use the import hook explicitly, arguably its greatest use case involves enabling Typeguard as a pytest plugin while running your test suite. Example 10-10 adds a Nox session that runs the test suite with runtime type checking.

Example 10-10. Nox session for runtime type checking with Typeguard

```
package = "random_wikipedia_article"

@nox.session
def typeguard(session: nox.Session) -> None:
    session.install(".[tests]", "typeguard")
    session.run("pytest", f"--typeguard-packages={package}")
```

Running Typeguard as a pytest plugin lets you track down type-safety bugs in a large codebase—provided it has good test coverage. If it doesn't, consider enabling runtime type checking for individual functions or modules in production. Be careful here: look for false positives from the type checks, and measure their runtime overhead.

Summary

Type annotations let you specify the types of variables and functions in your source code. You can use built-in types and user-defined classes, as well as many higher-level constructs, such as union types, `Any` for gradual typing, generics, and protocols. Stringized annotations and `Self` are useful for handling forward references. The `type` keyword lets you introduce type aliases.

Static type checkers like mypy leverage type annotations and type inference to verify the type safety of your program without running it. Mypy facilitates gradual typing by defaulting to `Any` for unannotated code. You can and should enable strict mode where possible to allow for more thorough checks. Run mypy as part of your mandatory checks, using a Nox session for automation.

Type annotations are available for inspection at runtime. They're the foundation for powerful features such as class generation with `dataclasses` or the `attrs` library, and automatic serialization and deserialization with the help of the `cattrs` library. The runtime type checker Typeguard allows you to instrument your code to verify the types of function arguments and return values at runtime. You can enable it as a pytest plugin while running your test suite.

There's a widespread sentiment that type annotations are for the sprawling codebases found at giant tech corporations—and not worth the trouble for reasonably sized projects, let alone the quick script you hacked together yesterday afternoon. I disagree. Type annotations make your programs easier to understand, debug, and maintain, no matter how large they are or how many people work on them.

Try using types for any Python code you write. Ideally, configure your editor to run a type checker in the background if it doesn't already come with typing support out of the box. If you feel that types get in your way, consider using gradual typing—but also consider whether there might be a simpler way to write your code that gives you type safety for free. If your project has any mandatory checks, type checking should be a part of them.

With this chapter, the book comes to a close.

Throughout the book, you've automated checks and tasks for your project using Nox. Nox sessions allow you and other contributors to run checks early and repeatedly during local development, in the same way they'd run on a CI server. For reference, here's a list of the Nox sessions you've defined:

- Building packages (Example 8-2)
- Running tests across multiple Python versions (Example 8-5)
- Running tests with code coverage (Example 8-9)
- Measuring coverage in subprocesses (Example 8-11)
- Generating the coverage report (Example 8-8)
- Locking the dependencies with uv (Example 8-14)
- Installing dependencies with Poetry (Example 8-19)

- Linting with pre-commit (Example 9-5)
- Static type checking with mypy (Example 10-7)
- Runtime type checking with Typeguard (Example 10-10)

There's a fundamental philosophy behind this approach, dubbed "Shift Left." Consider the software development lifecycle on a timeline extending from left to right—all the way from writing a line of code to running the program in production. (If you're Agile minded, picture the timeline as a circle where feedback from production flows back into planning and local development.)

The earlier you identify a software defect, the smaller the cost of fixing it. In the best case, you discover issues while they're still in your editor—their cost is near zero. In the worst case, you ship the bug to production. Before even starting to track down the issue in the code, you may have to roll back the bad deployment and contain its impact. For this reason, shift all your checks as far to the left on that imaginary timeline as possible.

(Run checks toward the right of the timeline as well. End-to-end tests against your production environments are invaluable for increasing confidence that your systems are operating as expected.)

Mandatory checks in CI are the main gatekeeper: they decide which code changes make it into the main branch and ship to production. But don't wait for CI. Run checks locally, as early as possible. Automating checks with Nox and pre-commit helps achieve this goal.

Integrate linters and type checkers with your editor as well! Alas, people haven't yet agreed on a single editor that everybody should use. Tools like Nox give you a common baseline for local development in your teams.

Automation also greatly reduces the cost of project maintenance. Contributors run a single command, such as `nox`, as an entry point to the mandatory checks. Other chores, like refreshing lock files or generating documentation, likewise only require simple commands. By encoding each process, you eliminate human error and create a basis for constant improvement.

Thank you for reading this book! While the book ends here, your journey through the ever-shifting landscape of modern Python developer tooling continues. Hopefully, the lessons from this book will remain valid and helpful, as Python continues to reinvent itself.

1 Jukka Lehtosalo, "Our Journey to Type Checking 4 Million Lines of Python", September 5, 2019.

2 "Specification for the Python Type System", last accessed January 22, 2024.

3 Tin Tvrtković, "Python Is Two Languages Now, and That's Actually Great", February 27, 2023.

4 In a future Python version, this will work out of the box. See Larry Hastings, "PEP 649 – Deferred Evaluation of Annotations Using Descriptors", January 11, 2021.

5 If you see an error message like "PEP 695 type aliases are not yet supported," just omit the `type` keyword for now. The type checker still interprets the assignment as a type alias. If you want to be more explicit, you can use the `typing.TypeAlias` annotation from Python 3.10 upward.

6 For brevity, I've removed error codes and leading directories from mypy's output.

7 As of this writing, the upcoming release of `factory-boy` is expected to distribute types inline.

8 Tim Peters, "PEP 20 – The Zen of Python", August 19, 2004.

9 As of this writing, mypy hasn't yet added support for PEP 695 type variables. If you get a mypy error, type-check the code in the Pyright playground instead or use the older `TypeVar` syntax.

10 In fact, the `cattrs` library is format-agnostic, so it doesn't matter if you read the raw object from JSON, YAML, TOML, or another data format.

11 If you're interested in this topic, you absolutely should read *Architecture Patterns in Python* by Harry Percival and Bob Gregory (Sebastopol: O'Reilly, 2020).

12 This is less useful than it may seem. `TypedDict` classes must list every field even if you use only a subset.

[13] If you call `check_type` directly, you'll need to pass the `collection_check_strategy` argument explicitly.